

Въведение в учебния предмет.

Структура на система за управление на бази данни (СУБД).

Общи бележки за Entity Relationship (ER) модел.

Анализиране и проектиране на релации.

Типове данни в MySQL.

Създаване на база данни с MySQL.

Въведение в учебния предмет

- **Какво представлява предметът по управление на бази данни?**
 - Целта на курса е да осигури теоретични и практични знания относно концепциите, методите и инструментите за работа с бази данни.
 - Учебният предмет разглежда процесите по събиране, съхранение, обработка и анализ на данни.
 - Включва както теоретични аспекти, така и практически приложения чрез работа с различни СУБД и MySQL.
- **Значение на Базите данни (БД):**
 - Без база данни е трудно да се управляват огромни обеми от информация.
 - Системите за управление на бази данни (СУБД) осигуряват механизми за съхранение, извличане, манипулиране и сигурност на данни.

Релационни бази данни

Релационните бази данни са вид бази данни, при които информацията се съхранява в таблици.

The diagram shows a table with four columns: CustomerID, FirstName, BirthDate, and CityID. It contains four rows of data. Callouts point to specific parts: 'Колона' points to the BirthDate header, 'Ред' points to the first row, and 'Клетка' points to the cell containing 'August'.

CustomerID	FirstName	BirthDate	CityID
1	Brigitte	03/12/1975	101
2	August	27/05/1968	102
3	Benjamin	15/10/1988	103
4	Denis	07/01/1993	104

Всяка таблица е съставена от редове (записи) и колони (атрибути/свойства). Тези таблици са свързани помежду си чрез определени правила, което позволява ефективното съхранение и извличане на данни. Терминът "релационна база данни" за първи път е предложен през 1970 година от ученът Едгар Код.

Система за управление на база данни (СУБД)

Системата за управление на база данни (СУБД) е софтуер, който отговаря за поддържането и управлението на базата данни. Тя предоставя необходимите механизми за съхранение, обработка и защита на данни. Примери за популярни СУБД включват:

- Microsoft SQL Server
- MySQL
- Access
- Oracle

Предимства на релационните бази данни

Релационните бази данни предлагат няколко основни предимства за съхранение и управление на информацията:

1. **Намаляване на обема на данните:** чрез премахване на излишни повторения на данни, като се осигурява тяхната нормализация.

2. **Поддържане на коректност и цялостност на данните:** СУБД могат да налагат ограничения върху стойностите на данни (например, не се позволяват невалидни стойности), което предотвратява влизането на некоректна информация.
3. **Независимост на софтуера от информацията:** данните се съхраняват отделно от софтуера, което означава, че дори при повреда на софтуерния продукт, самите данни ще останат безопасни.
4. **Споделяне на данни:** множество различни софтуерни приложения могат да използват една и съща база данни. Съществуват високи нива на сигурност и контрол върху достъпа, като например:
 - Някои потребители могат само да четат данни (read-only).
 - Други могат да четат и променят данни, но само в определени таблици или схеми.
 - Администраторите имат пълни права за управление на базата данни.
5. **Редовни операции върху данни:** СУБД поддържат редовни операции като архивиране (backup) и възстановяване (restore), които гарантират сигурността на данните.

Структура на СУБД

Една типична СУБД има следната структура:

- **Сървър:** основният софтуер, който осигурява интерфейса за достъп до базата данни и за манипулиране на данни.
- **Сервизни програми:** програми, които се грижат за регулярни задачи като създаване на резервни копия, оптимизиране на производителността, и поддържане на статистика за използването на базата данни.

Релации и основни правила

В контекста на релационните бази данни, терминът **релация** се използва за означаване на таблица, която съдържа множество записи с общи атрибути. За да бъде базата данни коректна и ефективна, тя трябва да отговаря на следните основни правила:

1. **Уникално име на релация:** всяка таблица трябва да има уникално име в рамките на базата данни.
2. **Уникално име на атрибут:** всеки атрибут (колона) в една таблица трябва да има уникално име в рамките на същата таблица.
3. **Отсъствие на дублиращи редове:** в една таблица не могат да съществуват два идентични записа.

4. **Атомарни стойности:** стойностите в таблицата трябва да бъдат атомарни – т.е. всяка стойност трябва да бъде от един тип и не трябва да е резултат от изчисление.
5. **Няма строга подредба на записите:** записите в таблицата не трябва да имат фиксирана подредба.
6. **Първичен ключ:** всяка таблица трябва да има поне един уникален атрибут, наречен **първичен ключ**. Той трябва да гарантира, че всеки ред в таблицата може да бъде идентифициран еднозначно.

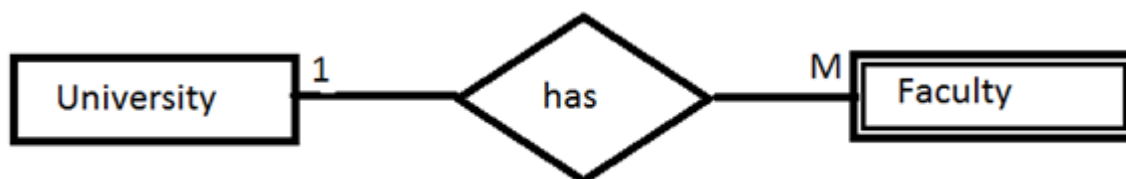
ER-диаграмите (Entity-Relationship диаграми) са инструмент за моделиране и проектиране на релационни бази данни. Те илюстрират обектите в дадена предметна област, техните свойства и връзките между тях.

Основни компоненти на ER-диаграмите:

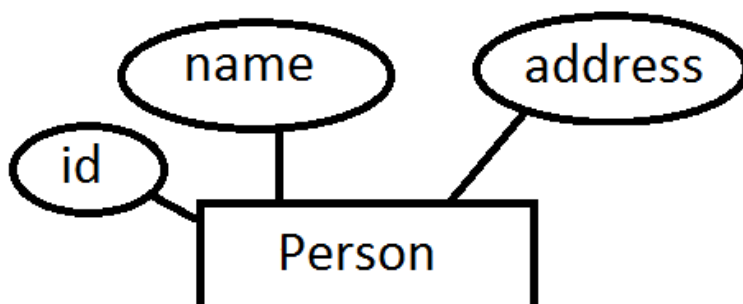
1. **Обект (Entity):** Представява самостоятелен елемент от предметната област, обозначен с правоъгълник. Примери: човек, студент, кола, къща.



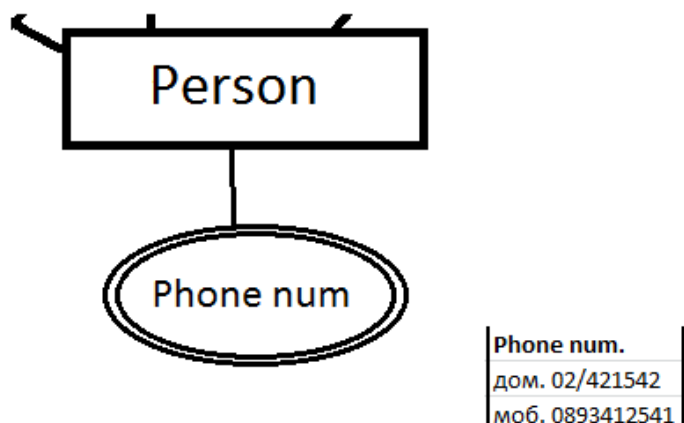
2. **Характеризиращ обект:** Обект, който не може да съществува извън определена предметна област. Например, във връзка с университети, "факултет" е характеризиращ обект, тъй като значението му е тясно свързано с университетската структура.



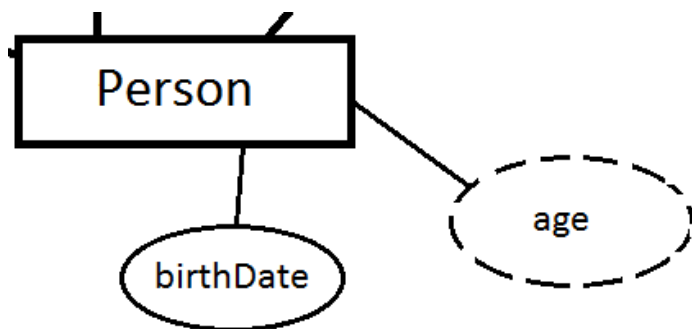
3. **Атрибути (Attributes):** Специфични свойства на обектите, обозначени с елипса. Пример: за обект "кола" атрибути могат да бъдат марка, модел, тип двигател.



4. **Атрибут с множество стойности (Multivalued attribute):** Атрибут, който има повече от една стойност, обозначен с двойна елипса. Пример: телефонен номер, който включва тип на телефона и самия номер.



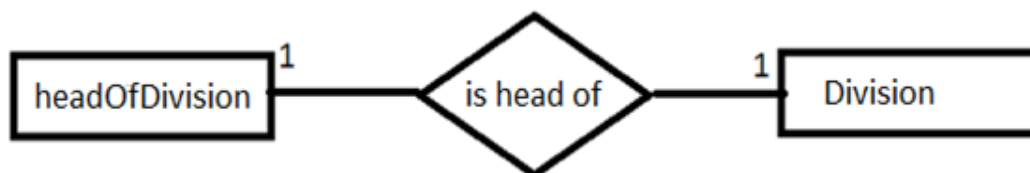
5. **Произведен атрибут (Derived attribute):** Атрибут, който може да бъде изведен от други атрибути на същия или друг обект, обозначен с прекъсната елипса.



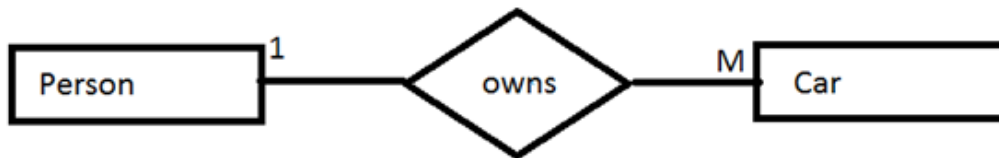
6. **Връзки (Relationships):** Представени като ромбове с името на връзката в средата. До релациите, които се свързват, се поставят 1, M, за да се покаже типът на връзката.

Видове връзки и тяхната кардиналност:

- **1:1 (One to One):** Един обект от едната релация е свързан с точно един обект от другата и обратно. Пример: един шеф управлява един отдел, и един отдел има точно един шеф.



- **1:M (One to Many):** Един обект от едната релация може да бъде свързан с множество обекти от другата. Пример: един човек притежава много коли; една кола се притежава от един човек.



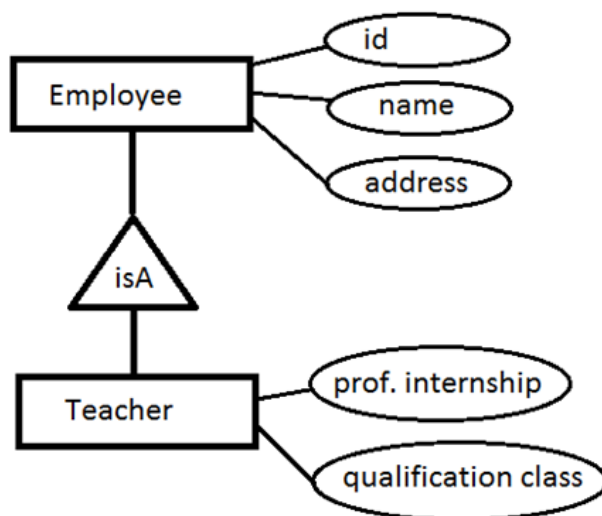
- **M:1 (Many to One):** Множество обекти от едната релация са свързани с един обект от другата. Пример: много работници се управляват от един мениджър; един мениджър управлява много работници.



- **M:M (Many to Many):** Множество обекти от едната релация са свързани с множество обекти от другата. Пример: един лекар лекува много пациенти; един пациент може да бъде лекуван от много лекари.



7. **Подклас обекти:** Обекти, които наследяват всички свойства на даден обект и добавят свои допълнителни атрибути. Връзката между тях се обозначава с триъгълник. Пример: обект "учител" е подклас на обект "работник", като освен общите атрибути (име, адрес) има и специфични като професионален стаж и клас квалификация.



ER-диаграмите са ценен инструмент при проектирането на бази данни, тъй като предоставят визуално представяне на структурата и връзките между данните, улеснявайки разбирането и планирането на системата.

Типове данни в релационните бази данни (SQL)

Въведение

Релационните бази данни съхраняват данните в таблици, които често са свързани помежду си. Всяка таблица (релация) има колони (атрибути) с предварително определен тип данни.

Различните системи за управление на бази данни (СУБД) предлагат различни типове данни, но в по-голямата си част те се припокриват.

SQL (Structured Query Language) е стандартен език за заявки към бази данни. Той е приет като стандарт от:

- **ANSI (American National Standards Institute) – 1986 г.**
- **ISO (Международна организация по стандартизация) – 1987 г.**

Една от най-популярните СУБД, базирани на SQL, е **MySQL**.

Основни типове данни в MySQL

Типовете данни в MySQL се делят на няколко категории:

1. **Числови типове (Numeric Data Types)**
2. **Текстови типове (String Data Types)**
3. **Типове за дата и час (Date and Time Data Types)**

1. Числови типове (Numeric Data Types)

Цели числа (Integer Types)

Тип	Диапазон (Signed)	Диапазон (Unsigned)	Макс. дължина (L)
TINYINT(L)	-128 до 127	0 до 255	1 байт (8 бита)
SMALLINT(L)	-32 768 до 32 767	0 до 65 535	2 байта
MEDIUMINT(L)	-8 388 608 до 8 388 607	0 до 16 777 215	3 байта
INT(L)	-2 147 483 648 до 2 147 483 647	0 до 4 294 967 295	4 байта
BIGINT(L)	-9 223 372 036 854 775 808 до 9 223 372 036 854 775 807	0 до 18 446 744 073 709 551 615	8 байта

Дробни числа (Floating-Point Types)

Тип	Описание	Диапазон	Точност
FLOAT(M,D)	Число с плаваща запетая	~ ±3.4E38	До 24 знака
DOUBLE(M,D) (или REAL)	Двойна точност	~ ±1.8E308	До 54 знака
DECIMAL(M,D) (или NUMERIC)	Фиксирана точност	Задава се от M и D	До 65 знака

Разлика между **DOUBLE** и **DECIMAL**:

- **DOUBLE** използва приблизителна стойност, което може да доведе до закръгляне.
- **DECIMAL** запазва точността, но работи по-бавно.

Логически (Булев) типове

- **BOOL** или **BOOLEAN** – Синоним на **TINYINT(1)**. Работи със стойности:
 - 0 → **FALSE**
 - 1 или друго число → **TRUE**

2. Текстови типове (String Data Types)

Фиксирана и променлива дължина

Тип	Описание	Макс. дължина
CHAR(L)	Фиксиран брой знаци (допълва с празни места)	1 – 255
VARCHAR(L)	Променлива дължина (заема толкова памет, колкото е нужно)	1 – 65 535

Разлика между **CHAR** и **VARCHAR**:

- **CHAR** винаги заделя фиксирана памет. Например **CHAR(10)** ще заема 10 байта, дори ако запишем „Test“ (4 символа).
- **VARCHAR** заема точно толкова място, колкото е нужно за съхранение на низа.

Големи текстови полета

Тип	Описание	Макс. дължина
TINYTEXT / TINYBLOB	До 255 символа	255 B
TEXT / BLOB	До 65 535 символа	64 KB
MEDIUMTEXT / MEDIUMBLOB	До 16 777 215 символа	16 MB
LONGTEXT / LONGBLOB	До 4 294 967 295 символа	4 GB

Разлика между **BLOB** и **TEXT**:

- **BLOB** се използва за двоични данни (например изображения).

- **TEXT** се използва за текстова информация (като дълги статии).

Изброими типове

Тип	Описание	Макс. стойности
ENUM('A', 'B', 'C')	Позволява избор само от предварително зададени стойности	65 535
SET('A', 'B', 'C')	Позволява записване на повече от една стойност	64

3. Типове за дата и час (Date and Time Data Types)

Тип	Формат	Диапазон
DATE	YYYY-MM-DD	1000-01-01 до 9999-12-31
DATETIME	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00 до 9999-12-31 23:59:59
TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970-01-01 до 2038-01-19
TIME	HH:MM:SS	-838:59:59 до 838:59:59
YEAR(4)	YYYY	1901 до 2155

Разлики между DATETIME и TIMESTAMP:

- **DATETIME** се използва за съхранение на дата и час без значение от часовата зона.
- **TIMESTAMP** е свързан с UTC и се променя спрямо часовата зона на сървъра.

Изборът на правилния тип данни е от ключово значение за производителността и ефективността на релационните бази данни.

- **За цели числа** използвайте INT, BIGINT, SMALLINT и т.н.
- **За дробни числа** предпочитайте FLOAT или DECIMAL според точността, която ви е нужна.
- **За текстови данни** използвайте VARCHAR за променливи дължини, TEXT за дълги текстове и BLOB за двоични файлове.
- **За дати и часове** изберете между DATE, DATETIME, TIMESTAMP в зависимост от нуждите ви.

Тази информация ще ви помогне да изберете най-подходящите типове данни за вашата база.

Релационни бази данни: ключове, модели и SQL команди

1. Основи на релационните бази данни

Релационните бази данни съхраняват информацията в таблици, като всяка таблица съдържа:

- **Колони (атрибути)** – определят типа на съхраняваните данни.
- **Редове (записи)** – представляват конкретни обекти.

За да се гарантира уникалността и взаимовръзките между данните, се използват **ключове**.

2. Видове ключове

2.1. Първичен ключ (Primary Key – PK)

Първичният ключ осигурява уникална идентификация на всеки запис в таблицата.

Характеристики:

- Всяка стойност в първичния ключ трябва да бъде **уникална**.
- Не може да съдържа **NULL** стойности.
- Обикновено се избира атрибут като **ID** или комбинация от няколко атрибута (съставен ключ).
- Пример: В таблица Person първичен ключ може да бъде EGN, защото всяко EGN е уникално.

2.2. Външен ключ (Foreign Key – FK)

Външният ключ създава връзка между две таблици.

Характеристики:

- Отнася се към **първичен ключ** на друга таблица.
- Гарантира **цялостност на данните** – не позволява стойности, които не съществуват в референтната таблица.
- Може да съдържа **NULL** стойности (ако връзката не е задължителна).
- Пример: В таблица Orders колоната customer_id може да бъде външен ключ, сочещ към id в таблица Customers.

3. Връзки между таблици

3.1. Връзка 1:1 (One-to-One)

- Всяка стойност в една таблица съответства на **точно една** стойност в друга таблица.
- Реализира се чрез външен ключ с **UNIQUE** ограничение.

- Пример: Passport и Person – всеки човек има само един паспорт.

3.2. Връзка 1:M (One-to-Many)

- Една стойност в първата таблица съответства на **няколко** стойности във втората.
- Реализира се чрез външен ключ в таблицата от страната „много“.
- Пример: Customers и Orders – един клиент може да има много поръчки.

3.3. Връзка M:M (Many-to-Many)

- Един запис в първата таблица може да бъде свързан с **много** записи във втората и обратното.
- Изисква **допълнителна свързваща таблица**, която съдържа два външни ключа.
- Пример: Students и Courses – студент може да посещава няколко курса, а един курс може да има много студенти.

4. Преобразуване от ER-модел към реляционен модел

При проектиране на база данни чрез ER-диаграма:

- **Обектите** се превръщат в **таблици**.
- **Атрибутите** на обектите стават **колони**.
- **Връзките между обектите** се реализират чрез **първични и външни ключове**.

5. Основни SQL команди

5.1. DDL (Data Definition Language) – структурна дефиниция

Тези команди определят **структурата** на базата данни.

Команда	Описание
CREATE DATABASE	Създаване на база данни
CREATE TABLE	Създаване на таблица
ALTER TABLE	Промяна на структурата на таблица
DROP DATABASE	Изтриване на база данни
DROP TABLE	Изтриване на таблица
CREATE INDEX	Създаване на индекс за оптимизация на заявки
DROP INDEX	Изтриване на индекс

5.2. DML (Data Manipulation Language) – обработка на данни

Тези команди работят с **данните в таблиците**.

Команда	Описание
INSERT	Вмъкване на нов запис
SELECT	Извличане на данни
UPDATE	Промяна на съществуващи данни
DELETE	Изтриване на записи

Релационните бази данни използват **ключове**, за да осигурят уникалност и взаимовръзка между таблиците. ER-моделите се преобразуват в **релационни структури**, а SQL командите позволяват създаване, управление и обработка на данните.

Тази структура прави информацията по-четима и по-ясно организирана.